ASHESI UNIVERSITY

ITEME FINANCE

APPLIED PROJECT

Applied Project submitted to the Department of Computer Science & Information Systems, Ashesi University in partial fulfilment of the requirements for the award of Bachelor of Science degree.

Daniel Byiringiro
COMPUTER SCIENCE

Eddy Kubwimana
COMPUTER SCIENCE

2025

DECLARATION

We hereby declare that this Applied Project is the result of our own original work and that no part of it has been presented for another degree in this university or elsewhere.

Candidate's Signature: .....

Candidate's Name: .....

Date: .....

Candidate's Signature: .....

Candidate's Name: .....

Date: .....

I hereby declare that preparation and presentation of this Applied Project was supervised in accordance with the guidelines on supervision of Applied Project laid down by Ashesi University.

Supervisor's Signature:

.....

Supervisor's Name:

.....

Date:

.....

Acknowledgement

First and foremost, we give thanks to God for His guidance, strength, and grace throughout the journey of building *Iteme Finance*. His presence has been a constant source of wisdom and perseverance, helping us overcome challenges and remain focused on our purpose of financial inclusion and empowerment.

We are especially grateful to Mr. Twagira Robert, whose support was key in connecting us to the Rwanda Stock Exchange. His efforts opened valuable doors that strengthened the practical grounding of our work. We also extend heartfelt appreciation to our mentor, Papa Kwame Anane, for his constant encouragement, critical feedback, and belief in our potential. His mentorship challenged us to think boldly and execute with integrity.

To the stakeholders who shared their insights and to the individuals, professionals, and diaspora members,who shared their experiences with us, we say thank you. Your voices shaped our design and vision. We also acknowledge the unwavering support of our families and friends. Lastly, we recognize the dedication of the entire *Iteme Finance* team; this project is a product of your commitment, collaboration, and shared dream for a more financially empowered Rwanda.

Abstract

Iteme Finance is a forward-thinking financial technology initiative committed to advancing financial inclusion in Rwanda. Our mission is to make investment opportunities more accessible, understandable, and tailored for employed individuals and Rwandans in the diaspora. We address the challenges of low financial literacy and limited access to capital markets through our three-pronged solution: an AI-powered investment advisor, automated portfolio management, and multilingual support for both English and Kinyarwanda users. To develop this solution, we conducted in-depth market research and stakeholder interviews, and engaged with the Rwanda Stock Exchange. This technical document outlines the architecture and implementation of our platform, highlighting key features such as broker and investor dashboards, stock exchange data integration, risk assessment, and financial education. This report presents our design methodology, system capabilities, testing processes, and recommendations for future growth, reinforcing our vision to democratize investing in Rwanda and beyond.

Table of Contents

DECLARATION **2**
Acknowledgement **3**
Abstract **4**
Table of Contents **5**
Chapter 1: Introduction **7**
Problem Background 7
Problem Significance 8
End Product 10
Chapter 2: Requirement Analysis **11**
Overview 11
Stakeholder Identification 11
Requirements Elicitation Methods 11
Appropriate Use Cases 12
Overview of Requirements 12
Functional Requirements 13
FR1: Web Interface: 13
FR2: AI-Powered Investment Advisory: 13
FR3: Multilingual Support: 13
FR4: Automatic Portfolio Management: 13
FR5: Financial Education: 13
Non Functional Requirements 14
NFR1: Performance: 14
NFR2: Scalability: 14
NFR3: Security: 14
NFR4: Usability: 14
NFR5: Reliability: 14
NFR6: Language Support: 14
Requirements Prioritization and Traceability 14
Conclusion 15
Chapter 3 : Architectural Design **16**
Software process 16
Database Design 16
Entity Design Overview 16
User Identity & Access Entities 16
Financial Instruments 17
Order Lifecycle Entities 17
Schema Design 17
Authentication Layer 17
Market Instrument Modeling 18

Order Tracking Mechanism....18
Relational Mapping....18
User–Order Relationships....18
Instrument–Order Relationships....18
Temporal Auditing....18
Security & Compliance Controls....19
Document Verification....19
Transaction Authorization....19
Data Integrity Constraints....19
Client-Server Architecture of the OneSpace Trading Platform....20
MVC Architecture....21
Presentation Layer....21
Business Logic Layer....21
Data Layer....21
Communication and Transport....22
Authentication, Authorization & Security....22
Process Map Architecture....22
User Onboarding and Identity Verification....23
Trading Lifecycle....24
Data workflow....24
Chapter 4: Implementation....25
Libraries....25
Tools....27
Programming Languages....27
Database implementation....27
Backend implementation & Frontend implementation....31
Investor login....32
Investor registration....33
Investor dashboard....34
Chapter 5: Testing and Results....37
Testing Methodology....37
Testing Results....38
Chapter 6: Conclusions and Recommendations....40
Conclusion....40
Recommendation....40
References....43

Chapter 1: Introduction

Problem Background

The Rwanda Stock Exchange (RSE) defines a stock exchange as an ”organized and regulated financial market where securities are bought and sold at prices governed by the forces of supply and demand” [12]. Stock exchanges establish trading rules, listing requirements and statutory requirements which are binding on all listed and trading parties [12]. The RSE was incorporated in 2005 but began operations on January 31, 2001 [12]. While neighboring exchanges such
as the Nairobi Securities Exchange have automated their operations by installing an automated trading and depository system [10] and the Dar es Salam Stock Exchange has automated its trading device in 2013, replacing open outcry trading [5] the RSE still conducts trading manually via open outcry and over-the-counter (OTC) [14].

Open outcry involves face-to-face communication on a physical trading floor, where traders announce their buy and sell intentions in a competitive, auction-like environment [15]. Over-the-Counter (OTC) on the other hand refers to a decentralized market where securities, such as stocks, bonds, and derivatives, are traded directly between parties through a broker-dealer network, instead of a centralized exchange like the RSE. This system allows companies that do not meet listing requirements of major exchanges to have their securities traded and provides a platform for small investors to enter the market, subject to certain regulations [8].

Manual stock exchanges like the RSE face a multitude of challenges such as being prone to human error [18], slow execution speeds [11], lack of transparency [17], limited accessibility, difficulty in price discovery [16], susceptibility to manipulation [16], and inability to handle large volumes of trades efficiently compared to electronic exchanges [6]. This makes them less efficient and potentially risky for investors and brokers. Specifically in Rwanda, the stock exchange’s

physical location limits brokers to those in the vicinity. Additionally, brokers have to manually manage the

portfolios of clients which is time consuming, error-prone and burdensome. Brokers also lack real time access to stock valuations and have to rely on daily trading reports given out by the RSE [4, 7].

Furthermore, brokers incur transportation costs to the stock exchange, which are often passed on to clients resulting in a higher service fee. Once at the exchange, brokers must manually place each order under pressure to meet client needs, a process that is both tedious and time-consuming. Investors also face challenges, as they are unable to access real time stock values and have to rely on periodic summaries from their brokers [9]. Moreover, there is no standardized communication channel for investors to engage with brokers, leaving them with limited ways to track their investments.

Problem Significance

The challenges facing the RSE are not just operational inefficiencies; they pose a serious risk on Rwanda's economic development and the growth of Rwanda's financial markets. As Rwanda strives to position itself as a hub for investment within East Africa, the manual nature of the RSE hampers its attractiveness to both local and international investors. This inefficiency limits capital formation, reduces liquidity and affects the country's ability to sustain long term development projects [7].

For brokers, the manual trading system increases operational costs, which are inevitably passed on to clients. These additional fees make the market less appealing to smaller investors, limiting financial inclusion. Over time, this could lead to a concentration of wealth among those who can afford these higher costs, exacerbating inequality in Rwanda's financial markets. Musonera and Safari [7] emphasize that automated exchanges are more transparent, efficient, and capable of handling higher trade volumes, which encourages broader participation.

For investors, the lack of real-time access to stock valuations and the absence of standardized communication with brokers hinders effective portfolio management. Investors are unable to make timely informed decisions, which increases the risk of financial losses. In an emerging market like Rwanda, where the stock market is a key avenue for raising capital for both public and private sectors, this lack of access to real time data undermines trust in the market. Mbituyimana [4] highlights that access to accurate and timely information is crucial for market efficiency, and its absence can lead to market manipulation, reduced trust, and ultimately, reduced market participation.

Additionally, the reliance on manual systems limits the scale of operations. Without automation, the RSE struggles to handle large volumes of transactions, which is essential for attracting institutional investors. As Rwanda seeks to grow its economy, the inability to modernize its financial infrastructure could lead to missed opportunities for both local businesses and international investors [9].

On a macro economic level, the stock exchange plays a critical role in economic development by facilitating the mobilization of capital for investment, which is essential for the growth of businesses and industries. For Rwanda, the Rwanda Stock Exchange (RSE) provides an avenue for companies and the government to raise capital through the sale of shares and bonds, connecting those with surplus funds to those with a deficit, thereby stimulating economic growth. The stock market helps to fund long-term development projects and ensures that savings are channeled into productive investments [9].

According to Cyuzuzo [2] an increase in market capitalization leads to an increase in GDP, demonstrating the significance of the stock market in the country's economic performance. Moreover, the stock exchange fosters financial stability by providing long-term funding sources. It reduces reliance on short-term borrowing and allows businesses to finance growth projects that

contribute to national development. This creates employment opportunities, improves infrastructure, and enhances overall economic resilience [4].

End Product

Our end product is OneSpace, a platform designed to promote financial inclusion in Rwanda. The platform aims to simplify access to the Rwanda Stock Exchange (RSE) for employed individuals and the Rwandan diaspora, providing a user-friendly experience through advanced technologies and artificial intelligence (AI). By making investments more accessible and understandable, OneSpace seeks to empower users to make informed financial decisions.

Our platform features a seamless web interface that enables users to easily manage their investment portfolios. Through this interface, users can buy, sell, and automate the trading of RSE-listed stocks, while also monitoring their portfolio performance in real-time. In addition, the platform integrates AI to offer personalized investment advice, using a large language model (LLM) trained on Rwandan financial data to simplify investment risk explanations, clarify financial jargon, and provide tailored investment recommendations based on user needs.

Iteme Finance also supports both English and Kinyarwanda, allowing users to interact with the platform in their preferred language. The platform includes speech-based interaction with the AI advisor and text-based queries, further enhancing accessibility. Moreover, the platform enables users to automate their investment strategies through customizable trading algorithms, scheduled portfolio rebalancing, and optimization aligned with personal financial goals. This feature ensures that users can manage their investments efficiently and according to their financial objectives.

In addition to its core investment functionalities, Iteme Finance provides financial education resources to help users build their financial literacy. These resources include simplified financial data and analytics, tutorials on best investment practices, and tools for improving financial knowledge.

Chapter 2: Requirement Analysis

Overview

The development of our platform, OneSpace, requires a thorough understanding of both user needs and system capabilities. The requirements gathering process focuses on identifying the core functionalities necessary for brokers and investors, who are the core users of our platform and key stakeholders such as the Rwanda Stock Exchange (RSE) and the Rwanda Capital Market Authority (RCMA).

Stakeholder Identification

The primary stakeholders in this project include:

- Brokers: Intermediaries facilitating stock trading on behalf of investors.
- Investors: Individuals and institutions looking to buy, sell, and manage stock portfolios.
- RSE: An exchange with the mission of providing the main platform for promotion of savings and raising funds for long term investment in Rwanda and beyond [13].
- RCMA: The Capital Market Authority of Rwanda, with a dual mandate, developing and regulating the capital markets industry, commodities exchange and related contracts, warehouse receipts systems and collective investment schemes [1].

Requirements Elicitation Methods

To ensure that the platform meets user expectations, a combination of methods was used for requirements gathering:

- Online Meeting: Communicating with the head of operation at the Rwanda Stock Exchange to understand how they currently perform and their view towards digitization.

- Interviews: Conducted with brokers and investors to understand their pain points with the current manual trading system.
- Surveys: Distributed to a broader range of potential users to capture general requirements.
- Market Analysis: Examined digital trading platforms in other countries to identify features that could be adapted for Rwanda's market.
- Regulatory Review: Ensured that all system requirements comply with the trading regulations of the Rwanda Stock Exchange.

Appropriate Use Cases

The appropriate use cases for OneSpace include:

- Brokers: Facilitating stock trades for investors, managing orders, and tracking market changes.
- Investors: Viewing real-time market data, buying and selling stocks, and managing portfolios.
- RSE: Monitoring trading activities, ensuring market integrity, and overseeing transactions.
- RCMA: Enforcing compliance with regulations, overseeing the capital markets, and managing reporting.

These use cases ensure that the platform addresses the needs of all stakeholders in an efficient and compliant manner.

Overview of Requirements

The requirements for OneSpace are categorized into functional and non-functional requirements. These requirements are aligned with the needs of investors, brokers, regulatory authorities, and Rwanda's economic development goals.

Functional Requirements

Functional requirements describe the core functionalities the system must provide to meet user needs.

FR1: Web Interface:

- Users can access and manage investment portfolios.
- Provide options to buy, sell, and automate trading of Rwanda Stock Exchange (RSE)-listed stocks.
- Enable real-time monitoring of portfolio performance.

FR2: AI-Powered Investment Advisory:

- Explain investment risks in simple terms.
- Clarify financial jargon.
- Provide tailored investment recommendations.

FR3: Multilingual Support:

- Support English and Kinyarwanda for all interactions.
- Enable speech-based interaction with the AI advisor.
- Facilitate text-based queries in both languages.

FR4: Automatic Portfolio Management:

- Allow users to create and apply customizable trading algorithms.
- Enable scheduled rebalancing of portfolios.
- Optimize strategies to align with personal financial goals.

FR5: Financial Education:

- Provide access to simplified financial data and analytics.
- Offer tutorials on investment best practices.
- Include tools for assessing and enhancing financial literacy.

Non Functional Requirements

Non-functional requirements define the quality attributes the system must achieve to deliver an optimal user experience.

NFR1: Performance:

- Ensure real-time updates for stock data and portfolio performance.
- Provide fast response times for AI-powered queries.

NFR2: Scalability:

- Support a growing number of users, including both individual and diaspora investors.
- Handle increased data loads from stock exchanges and user interactions.

NFR3: Security:

- Implement end-to-end encryption for all data exchanges and transactions.
- Conduct regular security audits to protect user data.

NFR4: Usability:

- Design an intuitive user interface for both the web and mobile platforms.
- Ensure accessibility for users with varying levels of financial literacy.

NFR5: Reliability:

- Provide high availability and minimal downtime for critical features.
- Implement robust error handling to ensure uninterrupted user experience.

NFR6: Language Support:

- Maintain accurate translations and speech recognition for English and Kinyarwanda.
- Continuously update the language model with relevant financial terminology.

All the above requirements will be addressed during the course of the project.

Requirements Prioritization and Traceability

The MoSCoW method is employed to prioritize requirements into:

- Must Have: Core trading functionalities, AI advisory, and portfolio management.
- Should Have: Multilingual support and advanced trading algorithms.
- Could Have: Additional financial education resources.
- Won't Have: Features irrelevant to current stakeholders or outside project scope.

All requirements are tracked using a requirements traceability matrix (RTM) to ensure alignment with project goals and stakeholders' needs.

Conclusion

This requirement analysis sets a clear foundation for the development of OneSpace, ensuring it addresses the challenges of manual stock trading in Rwanda. By prioritizing both functional and non-functional requirements, the platform is designed to provide an intuitive, secure, and scalable solution for brokers, investors, and regulatory authorities.

Chapter 3 : Architectural Design

Software process

The OneSpace platform is being developed to serve as a core financial infrastructure for Rwanda's capital market. Given the sensitive nature of trading activities, investor data, and fund transactions, our software process prioritizes security, reliability, and regulatory alignment throughout the lifecycle.

We follow an Agile approach as our development methodologies. Although we use Agile, we pay close attention to security and compliance practices in every module we develop to ensure high level security in both before and after integration of various modules . Each sprint cycle includes security checkpoints and audit trails integrated into our CI/CD workflows. We usually do two-week development sprints , weekly security-focused planning and design reviews and bi-weekly stand-ups and structured sprint retrospectives. We also keep both brokers, investor and Rwanda stock Exchange in loop to help us with continuous stakeholder feedback and compliance teams

Database Design

As the backbone of the OneSpace trading ecosystem, the database was engineered to handle core operational workflows such as user authentication, securities tracking, and real-time order processing. We implemented a normalized relational schema using PostgreSQL.

Entity Design Overview

The schema models the essential domain entities required to digitize and scale Rwanda’s capital markets infrastructure:

User Identity & Access Entities

- rse_employee: Represents official registry personnel, including login credentials and role metadata.
- broker: Captures licensed broker-dealer firms, KYC credentials, and digital verification tokens.
- investor: Stores individual investor profiles, national ID verification, and compliance data.

Financial Instruments

- securities: Central repository for tradable assets, including real-time market data, price history, and performance metrics.
- funds: Contains data on collective investment schemes and unit tracking.

Order Lifecycle Entities

- purchase_orders: Captures buy-side activity submitted through authorized brokers.
- sale_orders: Manages sell-side transactions.
- redemption_orders: Tracks investor-initiated fund redemption requests.

Schema Design

Several deliberate design choices were made to meet performance, auditability, and regulatory needs:

Authentication Layer

All user-related tables (employees, brokers, investors) include:

- Unique identifiers (UUIDs or prefixed codes) for traceability.
- Securely hashed password fields.
- Timestamps for audit trail generation.

Market Instrument Modeling

The securities table consists of over 20 fields capturing:

- Real-time market feeds (bid/ask/last prices).
- Trading activity (volume, turnover).
- Fundamental indicators (e.g., P/E ratio, dividend yield) for valuation insights.

Order Tracking Mechanism

Each order type table implements:

- Foreign keys to the broker and investor entities.
- Status flags (Pending,Completed, Rejected) with lifecycle timestamps.
- Specialized fields (e.g., security_id for purchase/sale, fund_id for redemptions).

Relational Mapping

All core relationships are explicitly enforced using foreign key constraints:

User–Order Relationships

- One broker → many orders (1:N)
- One investor → many orders (1:N)

Instrument–Order Relationships

- One security→ many purchase/sale orders (1:N)
- One fund → many redemption orders (1:N)

Temporal Auditing

- All entities include created_at fields.

- Order-related tables include updated_at fields with automatic timestamp refresh triggers.

Security & Compliance Controls

Security and compliance are integral to the schema:

Document Verification

- Broker entities include paths to encrypted license and certificate documents.
- Investors must provide verifiable national identification.

Transaction Authorization

- Orders require investor-side digital signatures to ensure non-repudiation.
- Broker authentication tokens are used to validate order origination.

Data Integrity Constraints

- Foreign key and cascade rules preserve referential integrity.
- NOT NULL and UNIQUE constraints are enforced on mission-critical attributes such as codes, symbols, and identifiers.

The finalized Entity-Relationship Diagram (Figure 1) outlines table relationships, indexing strategies, and access control logic. The schema is built to support high-throughput transactions while maintaining full transparency, traceability, and compliance with regulatory standards essential for Rwanda capital market modernization.

Figure1: *Entity-Relationship Diagram*

Client-Server Architecture of the OneSpace Trading Platform

The OneSpace platform adopts a modular, service-oriented client-server architecture designed to digitize and scale financial interactions within the Rwanda Stock Exchange (RSE) ecosystem. Our frontend application serves as the primary interface for investors, brokers, and RSE officials. It provides a performant, user-friendly environment for interacting with trading workflows and real-time market data.

In the initial phase, the web application serves as a functional prototype, enabling rapid validation of core assumptions and user pain points in Rwanda's capital market operations. This

approach ensures we build a solution grounded in real-world regulatory and user experience constraints.

MVC Architecture

The Onespace platform will be based on Model View Controller (MVC) software design pattern [3] three-layer structure: presentation layer, application layer, data layer. In addition to the three-layer structure, we will add a securities infrastructure layer.

Presentation Layer

The presentation layer is implemented using Next.js with server-side rendering (SSR) and static generation for optimal performance. The users (Investor, Broker, RSE Admin) access tailored dashboards and workflows and communicate with the backend using Communicates with backend services via secure HTTPS APIs.

Business Logic Layer

In Onespace platform, the business logic links the frontend and the data persistence layer by encapsulates core trading logic: order validation, fund management, compliance checks, and transactional auditing by protecting and enabling role-based access control (RBAC) and stateful session management.

Data Layer

The database design and implementation is backed by PostgreSQL, with schema managed through Drizzle ORM. The Drizzle ORM makes the manageability and scalability of Onespace databases easier. It optimizes migrations to production servers, transactional integrity, and referential consistency. In this layer, various APIs are exposed to render services through RESTful endpoints and internal service calls.

Communication and Transport

The communication between investor, broker or RSE happens when client-server interactions occur over the HTTP/1.1 application protocol secured by Transport Layer Security (TLS) to ensure end-to-end encrypted communication. The endpoints API's responses conform to a standardized JSON format to ensure consistent parsing and data consumption across client components and future integration with other third parties.

Authentication, Authorization & Security

One-space authentication is enforced via JWT-based tokens, email authentication with future optional multi-factor authentication Rwanda Stock Exchange employees and brokers. Session integrity is maintained with automatic token refresh and expiration controls and role-based access is enforced across all endpoints to ensure that only authorized actors can trigger sensitive actions such as order execution, fund redemptions and profile update.

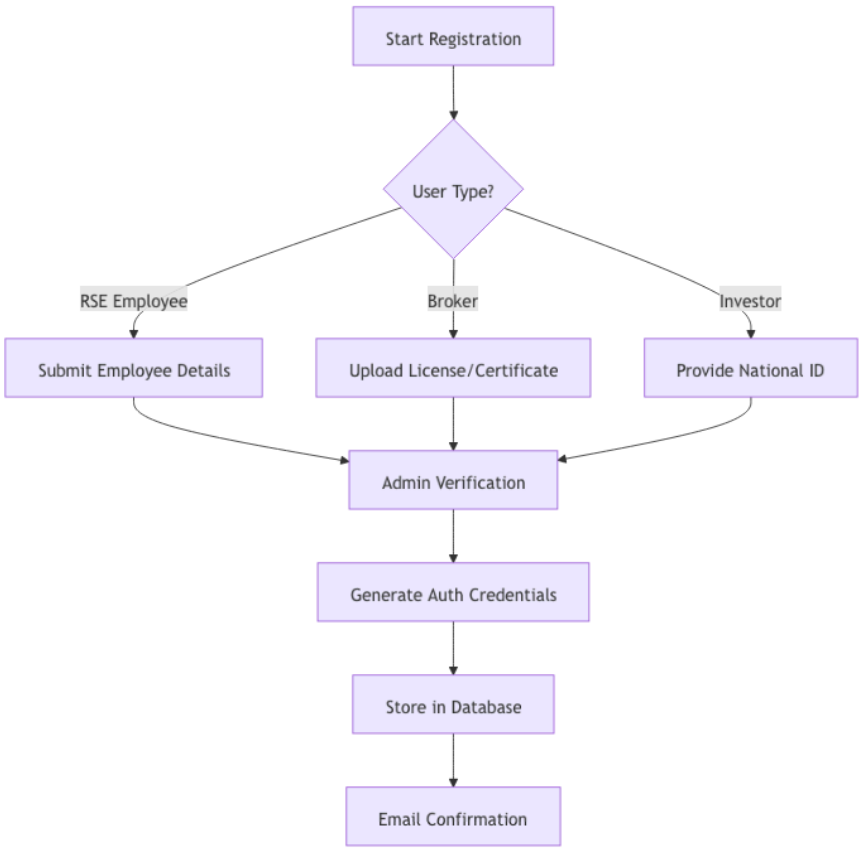
Process Map Architecture

OneSpace defines a set of foundational processes that govern how various user roles including investors, brokers, and RSE administrators interact with and derive value from the platform. At the heart of this system lies a secure and auditable access control mechanism; all users must complete a formal registration and authentication workflow before gaining access to any platform capabilities. This ensures regulatory compliance, data integrity, and trustworthiness across the ecosystem.

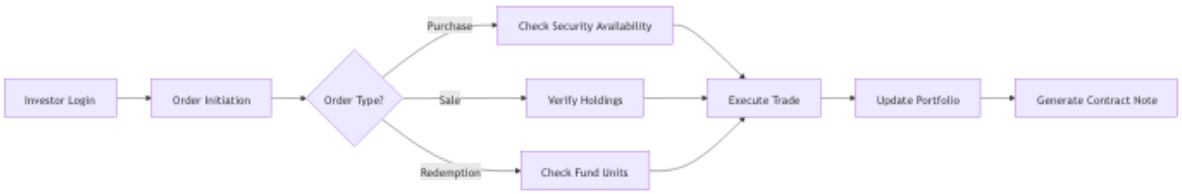
In the following architecture map, we outline the core process flows that underpin the OneSpace trading platform, including: User Onboarding and Identity Verification, Authentication and Session Management, Trading Lifecycle (Order Initiation to Execution), Data workflow. These

process maps are deliberately abstracted to highlight system-wide behaviors, role-specific transitions, and security checkpoints , forming the operational backbone of a compliant, high-availability trading environment tailored to the needs of Rwanda's capital markets.

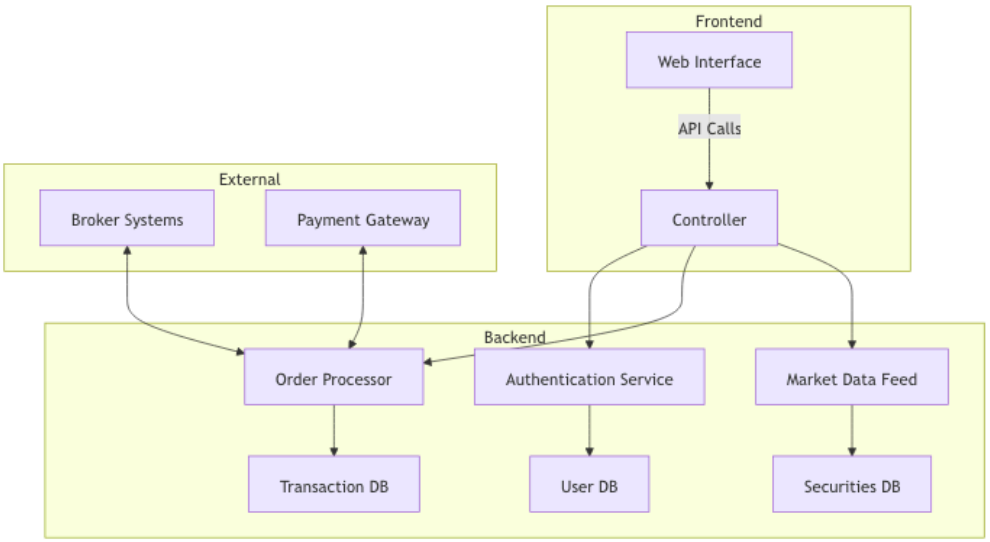
User Onboarding and Identity Verification



Trading Lifecycle



Data workflow



Chapter 4: Implementation

This chapter details the core technologies, architectural decisions, and engineering practices that underpin the implementation of the OneSpace trading platform. The web application was designed and developed with a focus on modularity, performance, and security, leveraging state-of-the-art tools and modern development paradigms to ensure scalability, reusability, and an exceptional user experience.

Libraries

At the heart of the OneSpace frontend lies Next.js, a React-based framework that supports both server-side rendering and static site generation. This hybrid capability allowed us to balance performance with dynamic content needs, critical for a trading environment that demands real-time updates without compromising load times. We adopted TypeScript across the codebase to ensure strong typing, improved developer experience, and better maintainability, especially as the application grows in complexity.

For crafting a refined and responsive user interface, we utilized Tailwind CSS, a utility-first framework that facilitated rapid development of consistent layouts and ensured mobile responsiveness without sacrificing design integrity. To further enhance the user experience, we integrated Radix UI, an accessibility-first library of low-level UI primitives, which enabled us to build highly interactive components.

Form handling and validation were engineered with precision, using React Hook Form for performant form state management. We enforced strict validation logic through Zod, a schema validation library that works seamlessly with React Hook Form via @hookform/resolvers. This pairing gave us both compile-time type safety and runtime validation, essential for processes like account registration, identity verification, and trade execution where input accuracy is non-negotiable.

On the data layer, OneSpace adopted Drizzle ORM, a type-safe and lightweight ORM built for modern TypeScript projects. This choice allowed us to define and manipulate our PostgreSQL schema directly within the application codebase while ensuring consistency between database logic and TypeScript types. Our backend services communicate with a Neon serverless PostgreSQL instance, enabling high concurrency and efficient scaling, particularly suited for a cloud-native financial application.

State management on the client side is handled through TanStack Query (react-query). This decision was driven by the need to efficiently manage server state, cache remote data, and handle background data synchronization with minimal boilerplate. For secure and authenticated API calls, we utilized Axios, configuring global interceptors for token injection and error handling. Our authentication system was built with NextAuth, a robust authentication solution that supports session management, OAuth providers, and role-based access control.

Given the nature of the platform, we implemented secure password handling using bcrypt, alongside form-level protections against injection and enumeration attacks. Additionally, Multer was integrated to support secure file uploads, such as user identification documents and financial certificates.To support effective communication with users, OneSpace integrates transactional email workflows using Resend, a modern email delivery service. Real-time in-app feedback is handled through a combination of notification libraries including React Toastify, React Hot Toast, and Sonner, each tailored for different user flows.

Development tooling and code quality were also a priority. The codebase adheres to strict linting rules through ESLint, with rules extended via eslint-config-next, which ensures code consistency and helps catch potential issues early in the development cycle. Utility libraries such as clsx, class-variance-authority, and cmdk contribute to a cleaner and more composable code structure, while libraries like js-cookie, libphonenumber-js, and country-data-list support seamless

internationalization and data integrity for global users.

Tools

For the development workflow and collaboration, we relied on a suite of modern tools that ensured efficiency, consistency, and streamlined deployment. Visual Studio Code served as our primary integrated development environment. For UI/UX design, we utilized Figma, which enabled rapid prototyping, real-time collaboration among team members. Git was used for version control, while GitHub acted as our central repository for code collaboration, issue tracking, and release management. To automate our deployment pipeline and maintain continuous integration and delivery , we configured GitHub Actions, which allowed us to build, test, and deploy the application automatically upon pushing updates to the repository.

Programming Languages

The development of the OneSpace platform was primarily done using TypeScript, a statically typed superset of JavaScript. TypeScript provided enhanced code safety, better tooling support, and improved developer productivity, especially in a large-scale application like OneSpace. By using TypeScript across both the frontend and backend, we ensured consistency, maintainability, and fewer runtime errors throughout the codebase.

Database implementation

During the implementation of OneSpace data layer, We used PostgreSQL as its primary relational database system, chosen for its robustness, scalability, and support for complex queries and data integrity. To manage and define the database schema in figure 1, we used Drizzle ORM, which provided a type-safe and flexible interface for interacting with the database.

```
import {pgTable, serial, text, integer, varchar, decimal, bigint, timestamp} from 'drizzle-orm/pg-core'

// rse table
export const $rseEmployee = pgTable('rseEmployee',
{
    id: serial('id').primaryKey(),
    fname: text('fname').notNull(),
    mname: text('mname').notNull(),
    lname: text('lname').notNull(),
    phone: text('phone').notNull(),
    email: text('email').notNull(),
    posit: text('posit').notNull(),
    code:  text('code').notNull(),
    password: text('password'),
    createdAt: timestamp('created_at').notNull().defaultNow(),
});


// broker table

export const $broker = pgTable('broker',
    {
        id: serial('id').primaryKey(),
        cname: text('cname').notNull(),
        ccode: text('ccode').notNull(),
        clicense_path: text('clicense_path').notNull(),
        phone: text('phone').notNull(),
        email: text('email').notNull(),
        ccertificate_path: text('ccertificate_path').notNull(),
        address: text('address').notNull(),
        auth_code: text('auth_code').notNull(),
        password : text('password'),
        createdAt: timestamp('created_at').notNull().defaultNow(),

    }
);
```

```
// investor broker
export const $investor = pgTable('investor',
    {
        id: serial('id').primaryKey(),
        fname: text('fname').notNull(),
        mname: text('mname').notNull(),
        lname: text('lname').notNull(),
        phone: text('phone').notNull(),
        email: text('email').notNull(),
        dbirth:  text('dbirth').notNull(),
        gender: text('gender').notNull(),
        nationality: text('nationality').notNull(),
        nationalId: text('nationalId').notNull(),
        code:  text('code').notNull(),
        password : text('password'),
        createdAt: timestamp('created_at').notNull().defaultNow(),

    });


// fund
export const $funds = pgTable("funds", {
        fund_id: serial("fund_id").primaryKey(),
        name: varchar("name", { length: 255 }).notNull(),
        symbol: varchar("symbol", { length: 10 }).notNull().unique(),
        net_asset_value: decimal("net_asset_value", { precision: 15, scale: 2 }),
        currency: varchar("currency", { length: 10 }).default("RWF"),
    });
```

```
// securities table
export const $securities = pgTable("securities", {
        security_id: serial("security_id").primaryKey(),
        symbol: text("symbol").notNull().unique(),
        name: varchar("name", { length: 255 }).notNull(),
        security_type: varchar("security_type", { length: 50 }).notNull(),
        exchange: varchar("exchange", { length: 50 }).notNull(),
        sector: varchar("sector", { length: 100 }),
        currency: varchar("currency", { length: 10 }).default("RWF"),
        market_cap: decimal("market_cap", { precision: 20, scale: 2 }),
        outstanding_shares: bigint("outstanding_shares", { mode: "number" }),
        dividend_yield: decimal("dividend_yield", { precision: 5, scale: 2 }),
        pe_ratio: decimal("pe_ratio", { precision: 10, scale: 2 }),
        last_price: decimal("last_price", { precision: 15, scale: 2 }),
        open_price: decimal("open_price", { precision: 15, scale: 2 }),
        high_price: decimal("high_price", { precision: 15, scale: 2 }),
        low_price: decimal("low_price", { precision: 15, scale: 2 }),
        prev_close_price: decimal("prev_close_price", { precision: 15, scale: 2 }),
        bid_price: decimal("bid_price", { precision: 15, scale: 2 }),
        ask_price: decimal("ask_price", { precision: 15, scale: 2 }),
        bid_volume: bigint("bid_volume", { mode: "number" }),
        ask_volume: bigint("ask_volume", { mode: "number" }),
        volume: bigint("volume", { mode: "number" }),
        change_percent: decimal("change_percent", { precision: 6, scale: 2 }),
        last_trade_time: timestamp("last_trade_time"),
        updated_at: timestamp("updated_at").defaultNow().$onUpdate(() => new Date()),
    });
```

```
// purchase order table
export const $purchase_orders = pgTable("purchase_orders", {
        order_id: serial("order_id").primaryKey(),
        broker_id: integer("broker_id").notNull().references(() => $broker.id),
        investor_id: integer("investor_id").notNull().references(() => $investor.id),
        security_id: integer("security_id").notNull().references(() => $securities.security_id),
        quantity: integer("quantity").notNull(),
        price_per_unit: decimal("price_per_unit", { precision: 15, scale: 2 }).notNull(),
        total_price: decimal("total_price", { precision: 20, scale: 2 }).notNull(),
        order_status: varchar("order_status", { length: 50 }).default("Pending"),
        order_type: varchar("order_type", { length: 20 }).default("Market"),
        order_instruction:text("order_instruction"),
        investor_signature:text('investor_signature').notNull(),
        created_at: timestamp("created_at").defaultNow(),
        updated_at: timestamp("updated_at").defaultNow().$onUpdate(() => new Date()),
    });

// sale order table

export const $sale_orders = pgTable("sale_orders", {
    order_id: serial("order_id").primaryKey(),
    broker_id: integer("broker_id").notNull().references(() => $broker.id),
    investor_id: integer("investor_id").notNull().references(() => $investor.id),
    security_id: integer("security_id").notNull().references(() => $securities.security_id),
    quantity: integer("quantity").notNull(),
    price_per_unit: decimal("price_per_unit", { precision: 15, scale: 2 }).notNull(),
    total_price: decimal("total_price", { precision: 20, scale: 2 }).notNull(),
    order_status: varchar("order_status", { length: 50 }).default("Pending"),
    order_type: varchar("order_type", { length: 20 }).default("Market"),
    order_instruction:text("order_instruction"),
    investor_signature:text('investor_signature').notNull(),
    created_at: timestamp("created_at").defaultNow(),
    updated_at: timestamp("updated_at").defaultNow().$onUpdate(() => new Date()),
});
```

```
// redemption order table
export const $redemption_orders = pgTable("redemption_orders", {
    order_id: serial("order_id").primaryKey(),
    broker_id: integer("broker_id").notNull().references(() => $broker.id),
    investor_id: integer("investor_id").notNull().references(() => $investor.id),
    fund_id: integer("fund_id").notNull().references(() => $funds.fund_id),
    units_redeemed: decimal("units_redeemed", { precision: 15, scale: 2 }).notNull(),
    price_per_unit: decimal("price_per_unit", { precision: 15, scale: 2 }).notNull(),
    total_redemption: decimal("total_redemption", { precision: 20, scale: 2 }).notNull(),
    redemption_status: varchar("redemption_status", { length: 50 }).default("Pending"),
    created_at: timestamp("created_at").defaultNow(),
    updated_at: timestamp("updated_at").defaultNow().$onUpdate(() => new Date()),
});



export type RSEEmployeeType = typeof $rseEmployee.$inferInsert;
export type Broker = typeof $broker.$inferInsert;
export type Investor = typeof $investor.$inferInsert;
export type Funds = typeof $funds.$inferInsert;
export type Securities = typeof $securities.$inferInsert;
export type Sale_orders = typeof $sale_orders.$inferInsert;
export type Purchase_orders = typeof $purchase_orders.$inferInsert;
export type Redemption_orders = typeof $redemption_orders.$inferInsert
```

Backend implementation & Frontend implementation

The frontend and frontend of the OneSpace platform was built using Next.js, a powerful React framework that supports server-side rendering and optimized performance out of the box. This choice allowed us to create a fast, scalable, and SEO-friendly web application tailored for complex user interactions such as registration, trading, and real-time updates. This full-stack approach allowed us to build and serve backend logic, such as authentication, trading workflows, and data management, directly within the same codebase as the user interface. By leveraging Next.js API routes with TypeScript, we ensure type safety, modularity, and tight integration between frontend components and backend services. This setup streamlined development, reduced complexity, and enabled faster iteration while maintaining strong security and performance standards for handling sensitive financial data.

Use case of Investor among our 3 types of users

Investor login

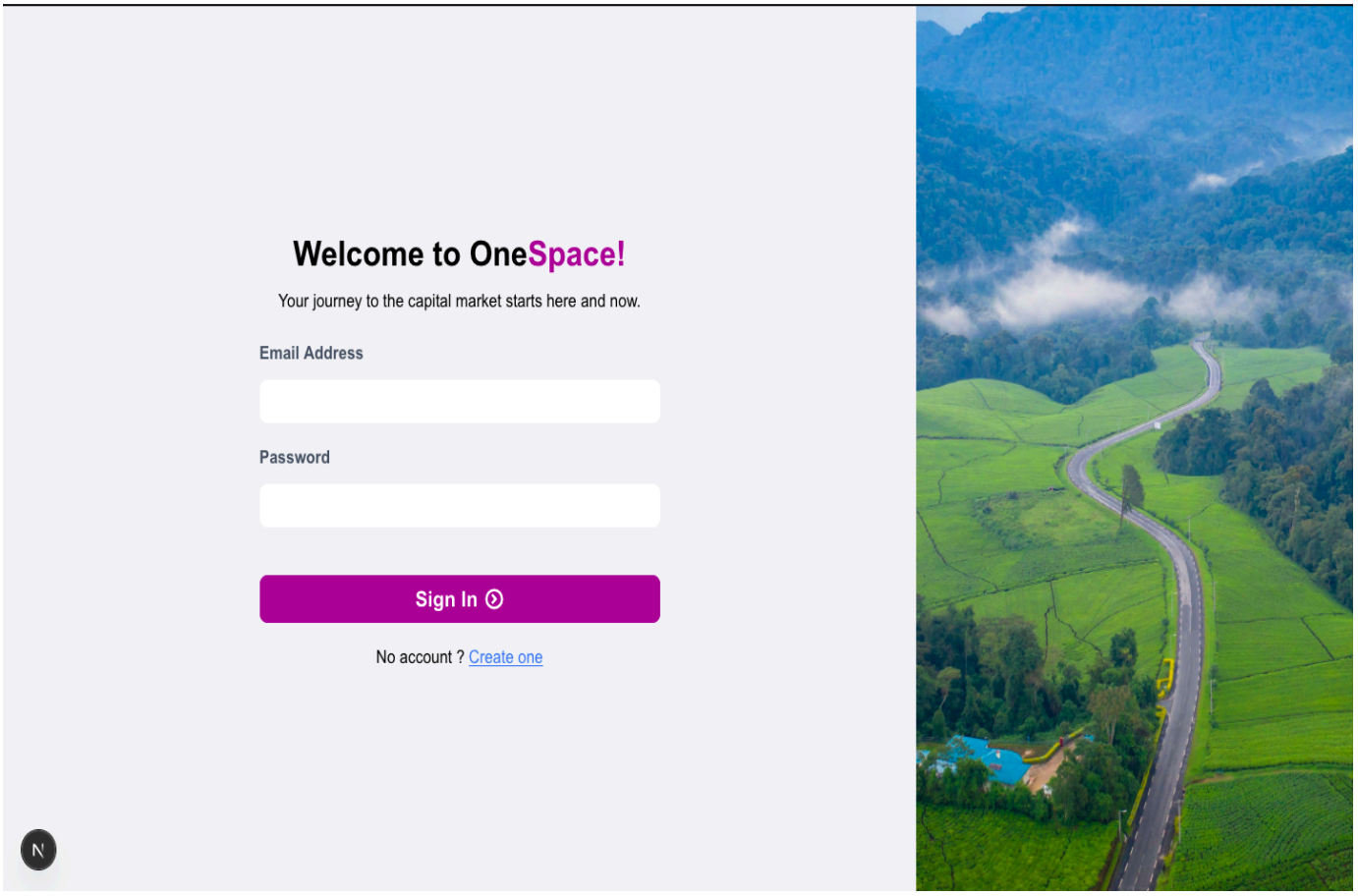
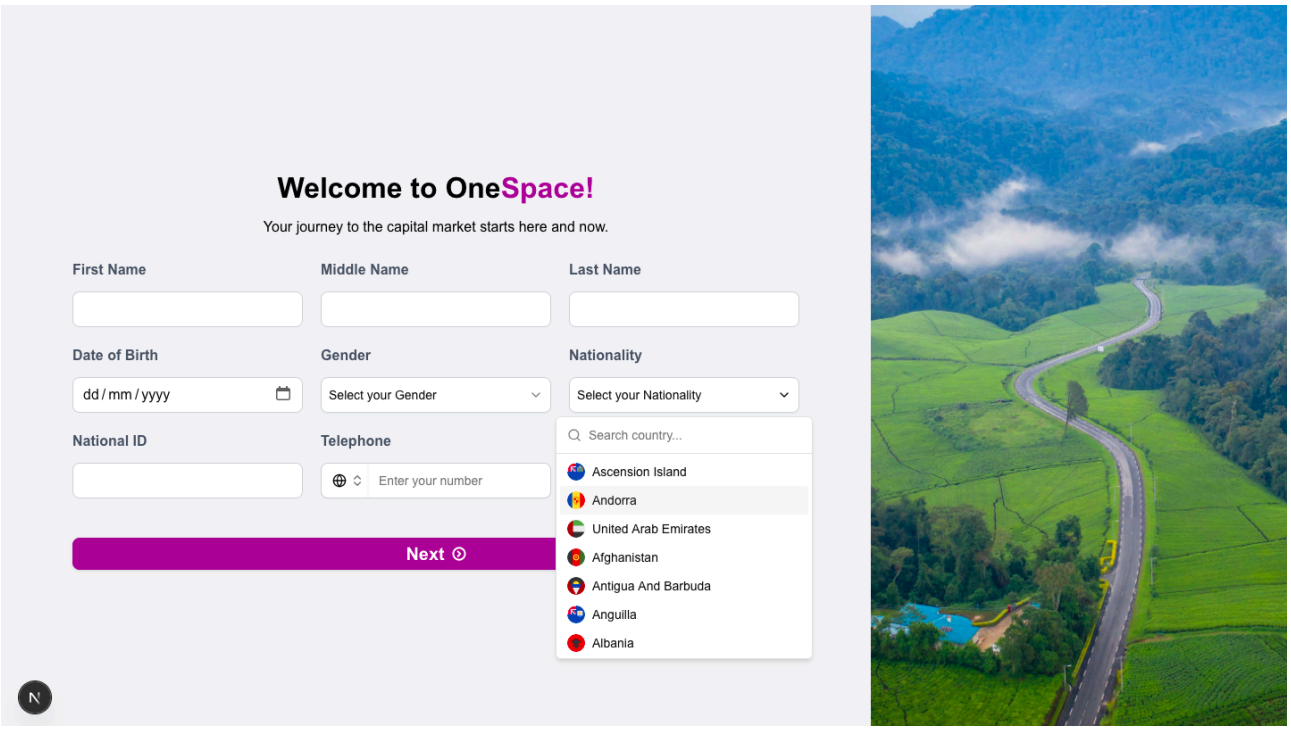
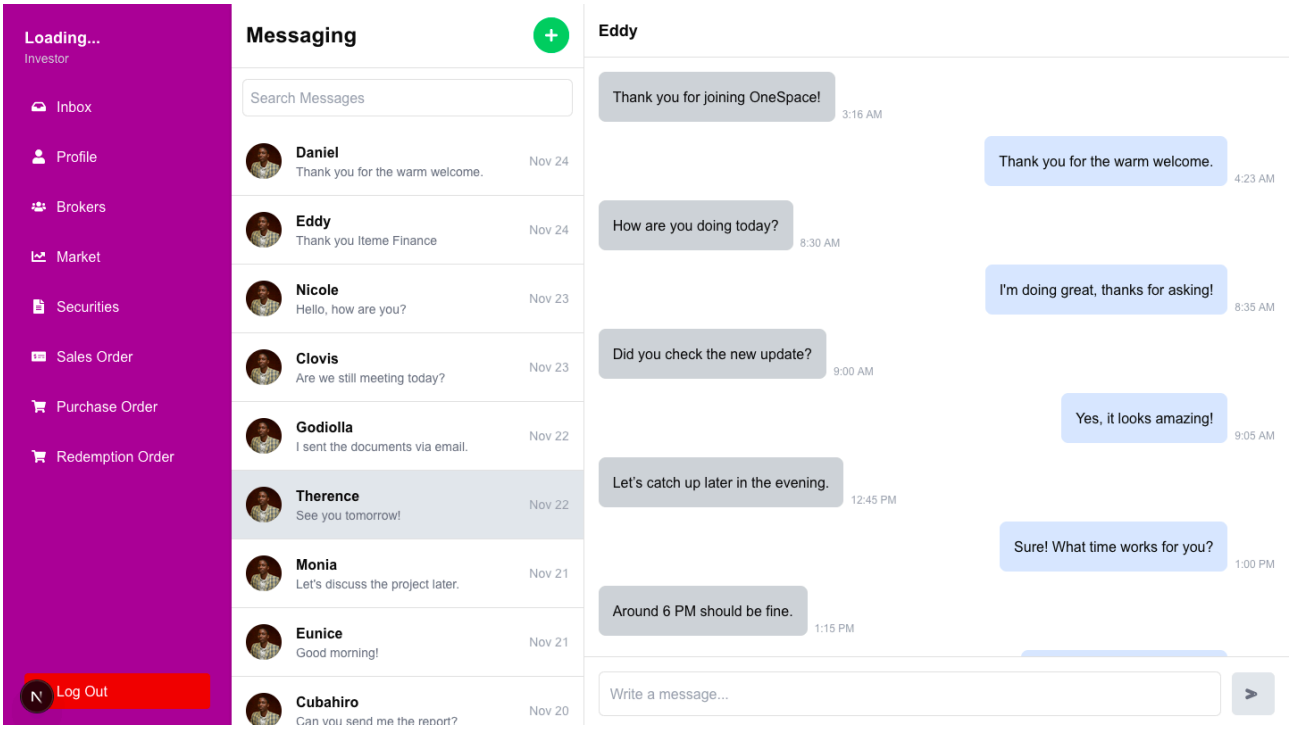


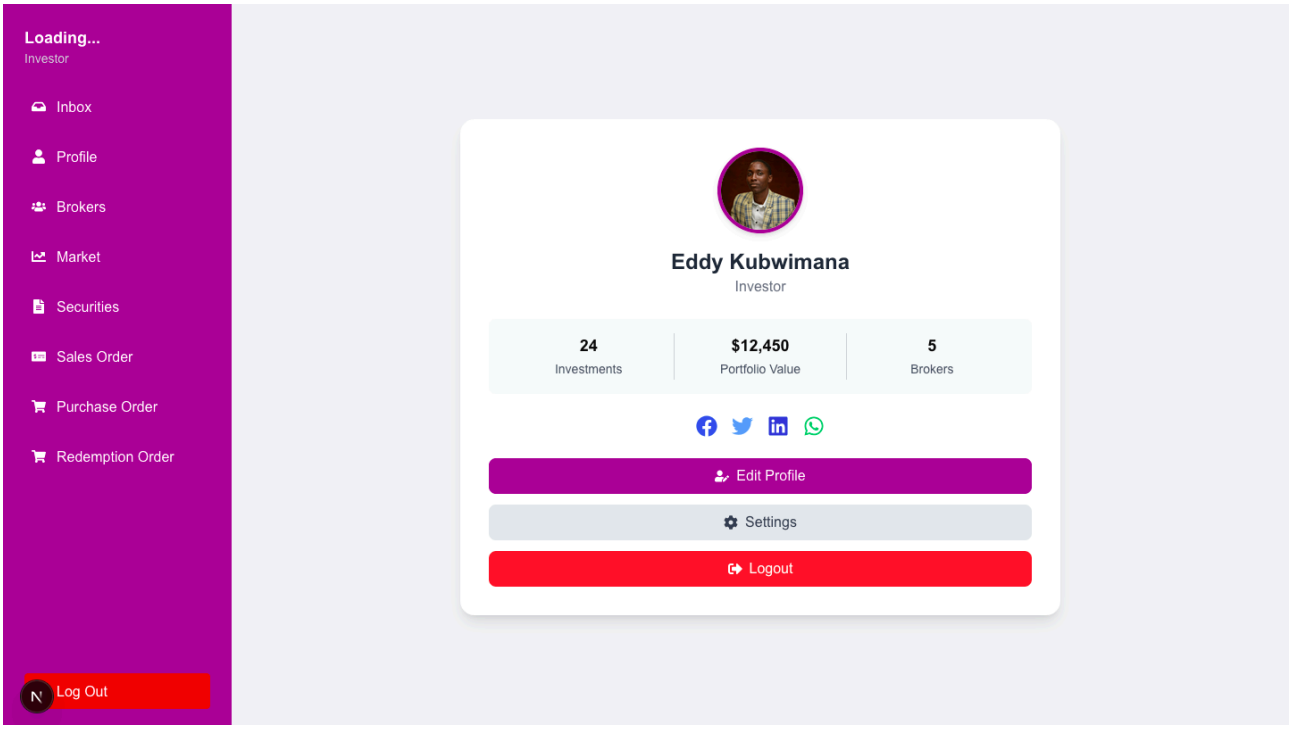
Figure 2: frontend for investor

Investor registration



Investor dashboard





Loading...
Investor

- Inbox
- Profile
- Brokers
- Market
- Securities
- Sales Order
- Purchase Order
- Redemption Order

Log Out

Rwanda Stock Exchange - Sale Order Form

Investor Details

Name: Eddy Kubwimana
Client No: 4
Phone: 0599346549
Email: agentconnectin@gmail.com

Orders Overview

Order ID	Security	Quantity	Price/Unit	Status	Actions
7	Crystal Telecom Ltd	400	100.00	Pending	
2	Bralirwa Plc	55	20.00	Pending	
8	Cimerwa Plc	120	100.00	Pending	
11	Cimerwa Plc	320	530.00	Pending	
12	I&M Bank Rwanda	340	570.00	Pending	

Loading...
Investor

- Inbox
- Profile
- Brokers
- Market
- Securities
- Sales Order
- Purchase Order
- Redemption Order

Log Out

Redemption Order

Iteme Finance - AC2344455

Broker Details

Name: Iteme Finance
Company: AC2344455
Phone: +233549034573
Office: 1 University Drive Ashesi

Fund Name

e.g. Rwanda Growth Fund

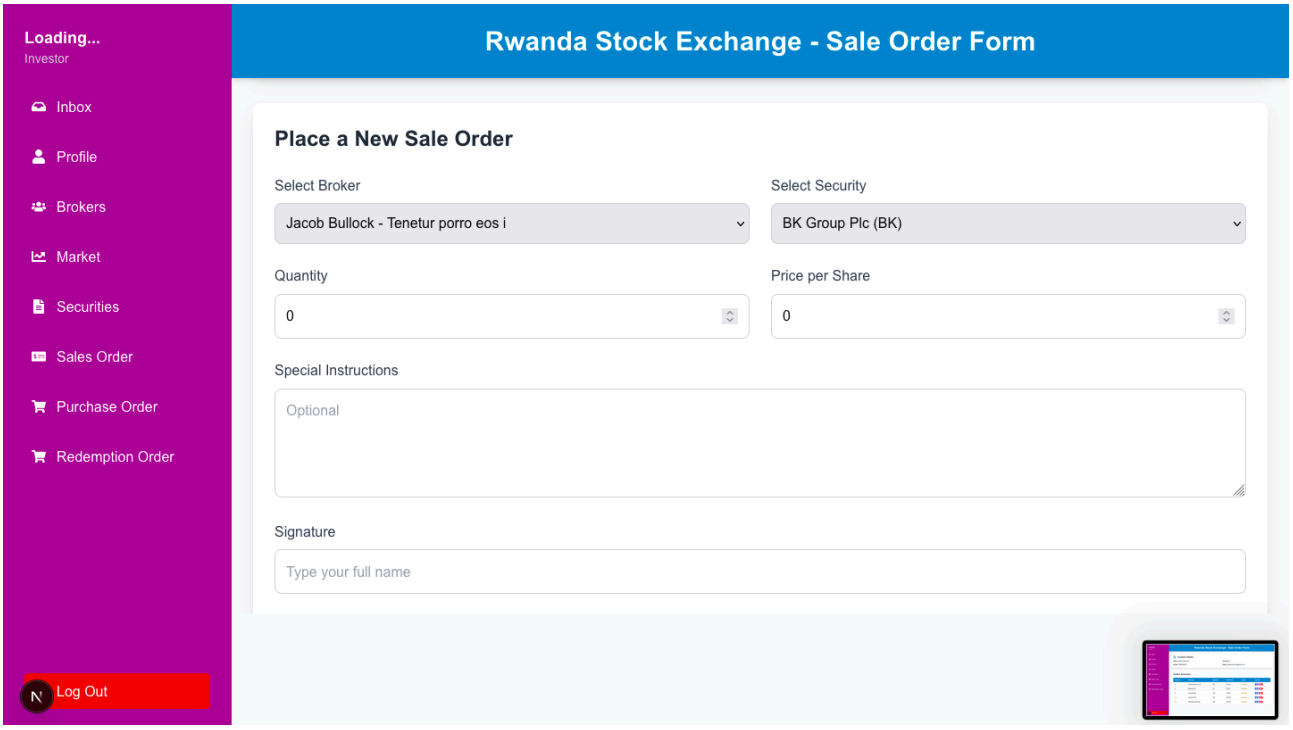
Units to Redeem

Expected Amount (RWF)

Settlement Bank Account

Bank Name

Branch Name


Loading...
Investor
Inbox
Profile
Brokers
Market
Securities
Sales Order
Purchase Order
Redemption Order
Log Out
Rwanda Stock Exchange - Sale Order Form
Place a New Sale Order
Select Broker
Jacob Bullock - Tenetur porro eos i
Select Security
BK Group Plc (BK)
Quantity
0
Price per Share
0
Special Instructions
Optional
Signature
Type your full name

Chapter 5: Testing and Results

Testing Methodology

The testing phase for our platform, OneSpace, was designed to evaluate the platform's functionality, performance, security, and usability. This comprehensive testing approach ensures that the platform meets the financial service requirements of its users and delivers a seamless experience. The testing process included unit testing, integration testing, system testing, user acceptance testing (UAT), and performance testing.

- **Unit Testing:** Focused on individual modules such as the investor dashboard, user profile management, authentication, and RSE (Rwanda Stock Exchange) data integration. Each module was tested in isolation to verify correctness and detect early defects during development.
- **Integration Testing:** Assessed how core modules interact, including portfolio management, authentication, and market data retrieval. This ensured the components functioned cohesively when responding to user queries or market fluctuations.
- **System Testing:** Performed after frontend and backend integration. Tests covered functionality (e.g., accurate trade execution), non-functionality (e.g., security protocols), and regression (e.g., verifying no impact from code updates). This validated that the platform met financial, UX, and compliance expectations.
- **User Acceptance Testing (UAT):** Conducted with target users, including Rwandan diaspora and salaried professionals. These users tested investment simulations, Kinyarwanda/English language toggles, and financial advice interactions. Feedback helped refine usability and align features with user goals.

- **Performance Testing:** Measured response time for portfolio updates, load time under user traffic, and efficiency of real-time financial data streaming. This ensured stable performance under varying network and usage conditions.

Testing Results

The testing phase provided valuable insights into the reliability and effectiveness of the OneSpace platform. Below is a summary of the outcomes for each testing stage:

- **Unit Testing Results:** All core modules passed unit tests with high reliability. Minor bugs were identified in the authentication module and were resolved before integration. Business logic for investment recommendations and financial literacy content retrieval performed accurately in isolation.
- **Integration Testing Results:** Components integrated smoothly, with APIs correctly handling data flow between modules. Minor latency was observed when switching between Kinyarwanda and English interfaces, which was optimized. The platform's backend effectively processed frontend requests, including trade execution and portfolio updates.
- **System Testing Results:** The full platform met all functional and non-functional requirements. Regression tests confirmed no adverse effects from recent updates. The platform adhered to financial compliance guidelines and delivered the expected features with minimal discrepancies.
- **User Acceptance Testing (UAT) Results:** 85% of users rated the platform as easy to use, especially the speech-based interactions and bilingual interface. Suggestions from users led to UI simplifications and improved onboarding flows. Key feedback indicated a need for clearer risk explanations, which has been addressed in the final release.
- **Performance Testing Results:** The platform maintained fast load times under simulated peak traffic (up to 50 concurrent users). Trade execution and data refresh occurred within

acceptable time frames (<1.2s). System responsiveness remained stable across varying network speeds.

Chapter 6: Conclusions and Recommendations

Conclusion

The Rwanda Stock Exchange (RSE) has long grappled with inefficiencies rooted in manual trading processes, including slow execution speeds, limited transparency, and accessibility barriers for investors and brokers. *Iteme Finance* addresses these challenges through *OneSpace*, a fintech platform designed to modernize Rwanda's capital markets by leveraging artificial intelligence, automation, and multilingual support.

The platform successfully integrates core functionalities such as real-time stock data access, AI-driven investment advisory, automated portfolio management, and bilingual interfaces (English and Kinyarwanda). Testing validated the platform's reliability, with 85% of users rating it as intuitive, particularly praising its speech-based interactions and seamless language toggling. Performance metrics confirmed the system's ability to handle concurrent users and deliver sub-second response times for critical operations like trade execution. By bridging gaps in financial literacy and democratizing access to the RSE, *OneSpace* aligns with Rwanda's broader economic goals of fostering financial inclusion, attracting investment, and stimulating long-term growth.

Recommendation

To ensure sustained impact and scalability, the following recommendations are proposed:

- **Technical Enhancements**
    - **Expand AI Training:** Incorporate additional datasets, including regional economic trends and global market shifts, to improve the accuracy of personalized investment recommendations.
    - **Optimize Multilingual Latency:** Further refine language-switching algorithms to eliminate residual latency observed during integration testing.

- **Blockchain Integration:** Explore blockchain technology to enhance transactional transparency and reduce susceptibility to market manipulation.

- **User Experience and Accessibility**
  - **Mobile Application Development:** Extend platform access by developing iOS and Android apps, prioritizing features like push notifications for real-time market alerts.
  - **Enhanced Risk Communication:** Implement visual aids (e.g., infographics, interactive simulations) to simplify risk explanations for users with limited financial literacy.
  - **Continuous Feedback Loops:** Establish mechanisms for ongoing user feedback to iteratively refine UI/UX design and onboarding processes.
- **Strategic Partnerships**
  - **Collaborate with Regulatory Bodies:** Partner with the RSE and Rwanda Capital Market Authority (RCMA) to advocate for policies supporting digital trading infrastructure.
  - **Regional Expansion:** Pilot integrations with neighboring exchanges (e.g., Nairobi Securities Exchange) to create a unified East African trading ecosystem.
  - **Financial Institution Partnerships:** Engage banks and microfinance institutions to offer bundled investment products, broadening user access to capital markets.
- **Scalability and Sustainability**
  - **Stress Testing:** Conduct large-scale performance tests to evaluate system stability under scenarios exceeding 1,000 concurrent users.
  - **Revenue Model Development:** Explore sustainable monetization strategies, such as tiered subscription plans or transaction fees, to ensure long-term platform maintenance.

- **Open-Source Contributions:** Share non-proprietary modules (e.g., Kinyarwanda NLP tools) with developers to foster innovation in Rwanda's fintech ecosystem.

- **Policy and Compliance**
  - **Regulatory Updates:** Establish a dedicated compliance team to monitor and adapt to evolving financial regulations in Rwanda and the East African Community.
  - **Advocacy for Automation:** Lobby policymakers to incentivize automation adoption among brokers, reducing reliance on manual processes.

By addressing these recommendations, *Iteme Finance* can solidify its role as a catalyst for Rwanda's financial transformation, ensuring that *OneSpace* remains at the forefront of inclusive, efficient, and transparent capital market solutions.

References

[1] Capital Market Authority, ”About CMA,” Capital Market Authority of Rwanda, Dec. 2024. [Online]. Available: https://www.cma.rw/index.php?id=12. [Accessed: Dec. 06, 2024].

[2] Cyuzuzo, G. U., ”Factors influencing the development of Capital markets in Rwanda: A case study of the Rwanda Stock Exchange,” 2018. [Online]. Available: DOI. [Accessed: Dec. 05, 2024].

[3] GeeksforGeeks, “MVC Framework - Introduction,” Sep. 5, 2023. [Online]. Available: https://www.geeksforgeeks.org/mvc-framework-introduction/. [Accessed: Dec. 6, 2024].

[4] Jeanluc, M. J., ”Stock Exchange Market Capitalization and Financial Performance of Firms in Rwanda Stock Exchange: A Survey of Listed Firms in Rwanda,” Journal of Finance and Accounting, 2022. doi:10.53819/81018102t5057.

[5] Julius, V., ”The Dar es Salaam Stock Exchange: A Catalyst for Tanzania’s Financial Transformation,” Digest TZ, Sep. 16, 2023. [Online]. Available: https://www.digest.tz/the-dar-es-salaam-stock-exchange-a-catalyst-for-tanzanias-financial-transformation/. [Accessed: Dec. 05, 2024].

[6] Kanaryan, N., ”Modelling the Risk at the Central European Stock Exchange at times of Crisis,” Economic Thought Journal, pp. 70–83, 2004.

[7] Musonera, E., and Safari, V., ”Establishing a Stock Exchange in Emerging Economies: Challenge and Opportunities,” Development Economics eJournal, 2008.

[8] Murphy, C. B., ”Over-the-Counter (OTC) Markets: Trading and Securities,” Investopedia, Mar. 5, 2024. [Online]. Available: https://www.investopedia.com/terms/o/otc.asp. [Accessed: Dec. 5, 2024].

[9] Ndagijimana, J., and Mulyungi, D., ”The Role of Stock Market on Economic Development of Rwanda; A Case Study of Rwanda Stock Exchange,” 2018.

[10] Onyuma, S., ”Exchange Automation and Adaptive Efficiency at the Kenyan Securities Market,” European Journal of Business and Management, vol. 12, no. 15, 2020. doi:10.7176/ejbm/12-15-06.

[11] Peterffy, T., and Battan, D. M., ”Why Some Dealers and Exchanges Have Been Slow to Automate,” Financial Analysts Journal, vol. 60, pp. 15–22, 2004. doi: 10.2469/faj.v60.n4.2632.

[12] Rwanda Stock Exchange, ”About Us: Background,” [Online]. Available: http://web.archive.org/web/20240826152020/https://www.rse.rw/About-Us/Background/. [Accessed: 26-Aug-2024].

[13] Rwanda Stock Exchange, ”About Us: Vision Mission and Values,” [Online]. Available: http://web.archive.org/web/20240826152022/https://rse.rw/About-Us/Vision-Mission-and-Values/. [Accessed: 26-Aug-2024].

[14] Rwanda Stock Exchange, ”Product and Services: Trading-System,” [Online]. Available: http://web.archive.org/web/20240424161228/https://www.rse.rw/product-and-services/Trading-System/. [Accessed: 24-April-2024].

[15] Scott, G., ”Open Outcry: What it is, How it Works, Decline in Popularity,” Investopedia, May 7, 2022. [Online]. Available: https://www.investopedia.com/terms/o/openoutcry.asp. [Accessed: Dec. 5, 2024].

[16] Shan-shan, G., ”On Techno-Risks Bearing Rule of Stock Exchange Online,” 2007.

[17] Tahmasebi, N., Tahmasebi, S., Vakili, F., and Tahmasebi, D., ”Study money and funds market challenges and its impact on the risk of the stock exchange,” Journal of Ecophysiology and Occupational Health, vol. 16, pp. 88–97, 2017. doi: 10.18311/JEOH/0/15966.

[18] Bansal, G., Hassija, V., Chamola, V., Kumar, N., and Guizani, M., ”Smart Stock Exchange Market: A Secure Predictive Decentralized Model,” in Proc. IEEE Global Communications Conference (GLOBECOM), 2019, pp. 1–6. doi: 10.1109/GLOBECOM38437.2019.9013787.